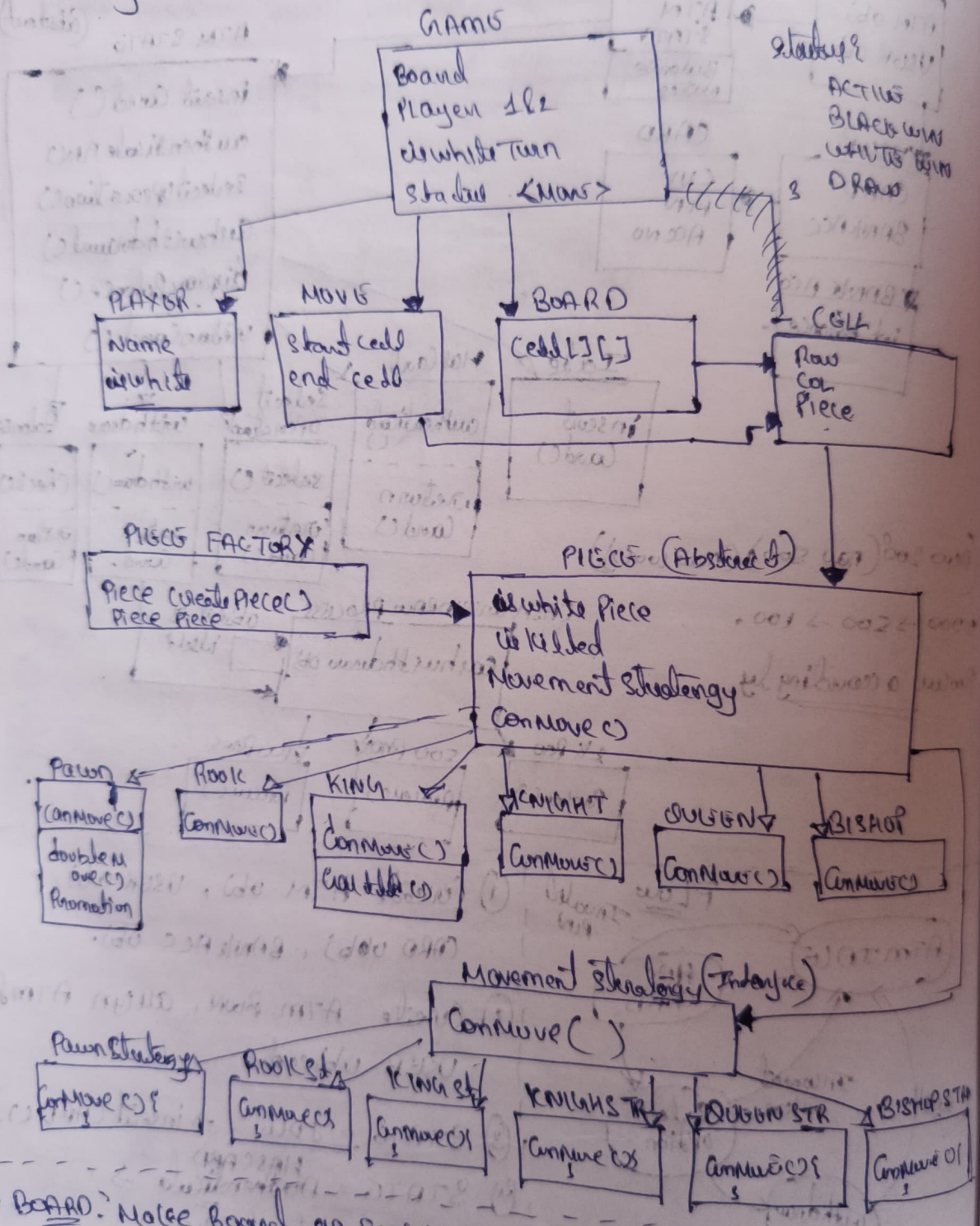
(10) CHESS GAME:

→ PIECE: King, Queen, Rook, Bishop, Pawn, Knight.

→ Board: 8x8 grid

→ player:



→ BOARD: Make Board as SINGLETON: so only 1 instance can be created throughout the game.

abstract Piece {
isWhite
killed = false
MovementStrategy
canMove() {
}; ms.canMove()

King extends Piece {
MovementStrategy
canMove() {
King (MovementStrategy) {
this.MoveSt = st
}
canMove() {
strategy.canMove()

interface
MovementStra {
boolean canMove()
}

KingStrategy impl
MovementStra
canMove() {
}

class Board {
Cell[][] board;
Board() {
Initialize()
}
}

class Move {
Cell start;
Cell end;
}

class Cell {
int row,
col
Piece Piece
}

abstract PieceFact {
static Piece CreatePiece(_) {
switch() {
case "King"
return new King
}
}

class Player {
name
isWhite;
}

Design Pattern used

1. Factory: used to create object based on Piece Type. No directly creating object
2. Strategy: Each Piece has different Move strategy. So, we created common MOVESTRATEGY, All Piece will extend this & implement CanMove()
3. SINGLETON: Create one object of Board.